perplexity

OOP 2 – Packages, Static, Singleton Class, In-built Methods (Detailed Notes)

These detailed notes are structured **by topic as explained in the video**, including code snippets, beginner explanations, and practical examples.

1. Introduction

- The video continues the Object-Oriented Programming (OOP) series in Java, focusing on practical concepts important for interview prep and understanding real-world Java code.
- Topics covered: **Packages**, **Static keyword**, **Singleton class**, and **in-built/static methods**.

2. Packages in Java

What is a Package?

- A **package** is a way to organize related classes and interfaces in Java, much like a folder organizes files on your computer.
- Helps to **avoid name conflicts** (e.g., two classes named `Main` in different packages) and makes management and maintenance of codebase easier.

How to Create and Use Packages

Example:

```
package com.kunal.introduction;

public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

- The file would be stored in folders: `com/kunal/introduction/HelloWorld.java`
- **Folder structure** mirrors the package structure.

Importing Classes from Packages

- To use a class from another package:

```
import com.kunal.messages.Message;

public class Main {
```

```
    public static void main(String[] args) {
        Message.printMessage();
    }
}
```

- Java's `import` **statement** fetches classes or entire packages.

Benefits of Packages:

- **Namespace Management:** Prevents naming clashes.
- **Modularity:** Break large projects into smaller units.
- **Access Protection:** Controlled class and method visibility.

3. Access Modifiers Recap

- Packages enable **encapsulation** and use of access modifiers:
  - `public`: Accessible everywhere.
  - *Default (no modifier)*: Accessible within the same package only.

4. Static Keyword in Java

Static Variables

- Declared with the `static` keyword.
- Shared **across all objects of the class** (class-level variable), not stored separately in each object.

Example:

```
class Human {
    String name;
    int age;
    static long population;

    Human(String name, int age) {
        this.name = name;
        this.age = age;
        population++;
    }
}
```

- Whenever a new `Human` object is created, `population` increases.
- Access `Human.population` directly without any object:

```
System.out.println(Human.population);
```

Static Methods

- Declared with `static`. Belong to the class, **not objects**.
- Invoked using class name, not object reference.

Example:

```
class Demo {
    static void printGreeting() {
        System.out.println("Hello from static method!");
    }
}
// Usage:
Demo.printGreeting();
```

Key Points about Static:

- Cannot use non-static variables/methods inside a static method directly.
- No `this` keyword inside static context.

5. Instance vs Static Members

Instance (Non-Static)	Static
Belongs to an object	Belongs to the class
Each object has its own copy	Shared (single copy) by all objects
Accessed using reference variable	Accessed using class name
Example: `kunal.name`	Example: `Human.population`

6. Real-life Example: Human Population

```
class Human {
    String name;
    int age;
    static long population;

    Human(String name, int age) {
        this.name = name;
        this.age = age;
        population++;
    }
}

public class Main {
    public static void main(String[] args) {
        Human h1 = new Human("Kunal", 22);
        Human h2 = new Human("Rahul", 21);
        System.out.println(Human.population); // Output: 2
```

```
    }
}
```

- All objects share the same `population` variable.

7. When to Use Static Variables or Methods

- **Static Variable:** When the value should be shared by all objects (e.g., tax rate, population, company name).
- **Static Method:** For utility functions (e.g., `Math.max()`), not dependent on object's data.

8. Singleton Class Concept

- Ensures **only one object** of a class exists during runtime.

Implementation Example:

```
class Singleton {
    private static Singleton instance;

    private Singleton() {} // private constructor

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

- `Singleton.getInstance()` returns the single instance of the class.

9. Built-in (Static) Methods

Math Class Example:

```
int maxVal = Math.max(7, 10);
double sqrtVal = Math.sqrt(25);
```

- Methods like `max`, `sqrt`, etc., are static, called with `Math.`

String Utility Methods:

```
String s = "hello";
System.out.println(s.toUpperCase()); // "HELLO"
int len = s.length(); // 5
```

- Learn to leverage Java's in-built classes for common operations.

10. Best Practices & Key Points

- Use packages to keep code organized and avoid naming conflicts.
- Use static variables when the property/behaviour should be common to all instances.
- Avoid using object reference with static methods/fields (use class name instead).
- **Singletons** are used where only one shared resource/object is needed (e.g., DB connection).
- Access control with packages helps encapsulate details and restrict access as needed.

11. Advanced Concepts Discussed

- **Accessing static fields before any object is created**.
- **Cannot access instance variables inside static context**.
- **Static initialization block** can initialize static variables:

```
class Example {
    static int count;
    static {
        count = 10;
    }
}
```

- **Static blocks** run once when the class loads.

12. Summary Table

Concept	Purpose/Usage	Example
Package	Code organization, prevent name conflicts	`package com.kunal.intro;`
Static var	Shared by all instances	`static long population;`
Static method	Utility, not dependent on object state	`Math.max(3,5);`
Singleton	Ensure one object only	`Singleton.getInstance()`

Practical Tip

- Experiment by creating different objects and manipulating static/instance variables.
- Use Eclipse or IntelliJ to observe package and class organization.

End of Notes – These notes summarize and explain all main concepts, code examples, and best practices taught in the video[1].

⁂

1. https://www.youtube.com/watch?v=_Ya6CN13t8k